

Sanskrit Document RAG System

Technical Report

Retrieval-Augmented Generation for Bhagavad Gita Corpus

Author: Ajay Mitkari

Date: July 2026

CPU-Only Inference | Hybrid Retrieval | Streamlit UI

Table of Contents

- 1. System Architecture and Flow
- 2. Details of Sanskrit Documents Used
- 3. Preprocessing Pipeline for Sanskrit Documents
- 4. Retrieval Mechanism
- 5. Generation Mechanism
- 6. Performance Observations
- 7. CPU Optimization Strategies
- 8. Conclusion

1. System Architecture and Flow

The Sanskrit Document RAG System follows a modular Retrieval-Augmented Generation architecture that separates document ingestion, retrieval, and generation into independent, composable components. This design ensures maintainability, testability, and the ability to swap components without affecting the overall pipeline.

Architecture Flow

The system operates in the following sequential stages:

- **Document Loader (ingest.py):** Downloads raw Sanskrit verse data and translations from a public GitHub repository in JSON format.
- **Preprocessor (ingest.py):** Merges verses with their transliterations, word meanings, and multi-language translations. Outputs a unified JSON database, TXT corpus, and PDF corpus.
- **Indexer (retriever.py):** Builds a character n-gram TF-IDF index over all verse chunks. Optionally builds a semantic vector index using multilingual sentence-transformers.
- **Retriever (retriever.py):** Accepts user queries and retrieves the most relevant verse chunks using hybrid TF-IDF + semantic search with Reciprocal Rank Fusion.
- **Generator (generator.py):** Constructs a context-grounded prompt with retrieved verses and generates a coherent response using a CPU-based LLM (Ollama or HuggingFace Transformers).
- **Web Interface (app.py):** A Streamlit application presenting a saffron-gold themed UI for interactive query-response with real-time metrics.

2. Details of Sanskrit Documents Used

The primary corpus consists of the **Srimad Bhagavad Gita**, one of the most important texts in Hindu philosophy and Sanskrit literature. The Gita is a 700-verse scripture that is part of the Indian epic Mahabharata.

Property	Details
Source Text	Srimad Bhagavad Gita
Language	Sanskrit (Devanagari script)
Total Chapters	18
Total Verses	~700
Data Format	JSON (verse text, transliteration, word meanings, translations)

Data Source	praneshp1org/Bhagavad-Gita-JSON-data (GitHub)
Translations	English (Swami Sivananda et al.), Hindi
Output Formats	JSON database, TXT corpus, PDF corpus

3. Preprocessing Pipeline for Sanskrit Documents

The preprocessing pipeline transforms raw JSON data into a structured, searchable corpus:

- **Data Download:** Raw verse data (verse.json) and translations (translation.json) are downloaded from the public GitHub repository using urllib.
- **Parsing and Merging:** Verses are matched with their corresponding translations by verse_id. English translations are prioritized (Swami Sivananda preferred), and Hindi translations are also extracted.
- **Text Normalization:** Whitespace is normalized, empty lines are removed, and text is cleaned for consistent indexing.
- **Chunk Strategy:** Each verse constitutes one retrieval chunk, containing all its metadata: Sanskrit text, transliteration, word meanings, and translations. This ensures that retrieved chunks are semantically complete.
- **Output Generation:** Three output formats are produced: (1) gita_processed.json for programmatic access, (2) bhagavad_gita_corpus.txt for readable text, and (3) bhagavad_gita_corpus.pdf as a formatted document.

4. Retrieval Mechanism

The retrieval system employs a hybrid approach combining keyword-based and optional semantic search to maximize recall and precision across Sanskrit, transliterated, and English queries.

4.1 TF-IDF Character N-Gram Retrieval

The primary retriever uses scikit-learn's TfidfVectorizer with a character n-gram analyzer (analyzer='char_wb', ngram_range=(2,5)). This design choice is critical for Sanskrit text retrieval because:

- Sanskrit transliterations have significant spelling variations (e.g., 'dharma' vs 'dharmma', 'karma' vs 'karmma').
- Character n-grams capture sub-word patterns that are resilient to these variations.
- The char_wb analyzer respects word boundaries, preventing false matches across words.
- Sublinear TF (sublinear_tf=True) dampens the effect of high-frequency terms.

4.2 Semantic Vector Retrieval (Optional)

When enabled, a multilingual sentence-transformer model (paraphrase-multilingual-MiniLM-L12-v2) encodes verse chunks and queries into dense vectors. Cosine similarity is used to find semantically similar verses, even when exact keyword matches are absent.

4.3 Reciprocal Rank Fusion (RRF)

When both retrievers are active, their results are combined using Reciprocal Rank Fusion with the formula: $RRF_score = \sum (1 / (k + rank_i))$ for each retriever i . This produces a balanced ranking that leverages the strengths of both approaches.

5. Generation Mechanism

The generation component supports two CPU-only inference backends:

5.1 Ollama Backend (Primary)

Ollama provides a lightweight, optimized runtime for running LLMs locally. The system uses the qwen2.5:0.5b model (500M parameters), which requires approximately 350MB of RAM and generates tokens at 5-15 tokens/second on CPU.

5.2 HuggingFace Transformers Backend (Fallback)

As a fallback, the system can load Qwen/Qwen2.5-0.5B-Instruct directly using the HuggingFace transformers library with PyTorch. This is fully self-contained but typically slower than Ollama on CPU.

5.3 Prompt Engineering

The generator constructs context-grounded prompts that include the retrieved verse context (Sanskrit text, transliteration, word meanings, and translations) along with a system instruction to act as a Sanskrit scholar. This ensures responses are grounded in the actual corpus and reduces hallucination.

6. Performance Observations

The following table summarizes expected performance characteristics on a typical CPU-only system (Intel Core i5, 8GB RAM):

Metric	Value	Notes
Retrieval Latency (TF-IDF)	50-200 ms	700 verse corpus, sparse matrix ops
Retrieval Latency (Semantic)	200-500 ms	Depends on model load time
Generation Latency (Ollama)	5-20 s	qwen2.5:0.5b, 100-200 tokens
Generation Latency (HF)	10-45 s	Qwen2.5-0.5B-Instruct, CPU float32

Total Pipeline Latency	5-45 s	Retrieval + Generation combined
Model Memory (Ollama)	~350 MB	Quantized model in Ollama runtime
Model Memory (HF)	~1.2 GB	Float32 weights in PyTorch
TF-IDF Index Size	~5 MB	Sparse matrix in memory
Corpus Size	~2 MB	700 verses with full metadata

7. CPU Optimization Strategies

- **Small Model Selection:** The 0.5B parameter Qwen2.5 model was chosen specifically for CPU inference. It fits comfortably in 350MB-1.2GB RAM while maintaining reasonable generation quality for scholarly text.
- **Ollama Runtime:** Ollama provides hardware-optimized inference with GGUF quantized models, typically achieving 2-3x speedup over raw PyTorch on CPU.
- **Sparse TF-IDF Indexing:** The TF-IDF matrix uses scipy sparse matrices, ensuring near-zero memory overhead for the retrieval index.
- **Character N-Gram Efficiency:** Character-level indexing with max_features=50000 caps vocabulary size while maintaining retrieval quality.
- **Streamlit Caching:** The @st.cache_resource decorator ensures the pipeline (model + index) is loaded only once per session.
- **Optional Semantic Search:** Semantic vector search is disabled by default and can be toggled on only when sufficient RAM is available.

8. Conclusion

This project demonstrates a complete, modular Retrieval-Augmented Generation system for Sanskrit documents, operating entirely on CPU-based inference. The system successfully ingests, indexes, retrieves, and generates responses from the Bhagavad Gita corpus using a combination of character n-gram keyword search and optional semantic vector retrieval.

Key achievements include: (1) robust handling of Sanskrit text in both Devanagari and transliterated formats, (2) efficient CPU inference with sub-minute response times, (3) a modular architecture that allows easy swapping of retrieval and generation components, and (4) a polished web interface for interactive exploration.

Future Improvements

- Support for additional Sanskrit texts (Upanishads, Vedas, Arthashastra).
- Fine-tuned Sanskrit-specific embedding models for improved semantic retrieval.

- GPU-accelerated inference for larger models (7B+ parameters).
- Multi-turn conversational interface with chat history.
- Devanagari OCR for ingesting scanned Sanskrit manuscripts.